Module – 3: Shortest Paths

Dr. Nirnay Ghosh

Shortest Path Problems

- **Weighted graphs**: Graphs that have a number assigned to each edge.
- Used to model airline systems, computer networks, etc.
- Airline system models – distance, fare, travel time
- Computer network models – distance, response time, lease rate.
- Two problems involving weighted graphs:
  - Determining a path of least *length* (different from the number of edges in a path in a graph without weights) between two vertices in a network (E.g., what is a shortest path in air distance between Boston and Los Angeles? What is the cheapest fare between these two cities? Which set of telephone lines gives a fastest response time for communications between San Francisco and New York?)
  - Determining a circuit of shortest total length that visits every vertex of a complete graph exactly once (E.g., traveling salesman problem)

Shortest Path Problems (*Contd...*)

- **Input**
  - Directed graph $G = (V, E)$
  - Weight function $w : E \rightarrow R$
- **Weight of path** $p = < v_0, v_1, \ldots, v_k >$
  - $w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$
  - Shortest-path weight from u to v:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \overset{p}{\rightsquigarrow} v\} & \text{if there is a path from } u \text{ to } v \text{ ,} \\ \infty & \text{otherwise .} \end{cases}$$

A ***shortest path*** from vertex u to vertex v is then defined as any path p with weight $w(p) = \delta(u, v)$.

- Edge weights can represent metrics other than distances, such as time, cost, penalties, loss, or any other quantity that accumulates linearly along a path and that we want to minimize.

Variants

- *Single-source shortest-paths problem*: given a graph $G = (V, E)$, find a shortest path from a given source vertex $s \in V$ to every vertex $v \in V$.
- The algorithm for the single-source problem can solve many other problems, including the following variants.

- **Single-destination shortest paths**
  - Find a shortest path to a given destination vertex t from each vertex v.
  - Reversing the direction of each edge $\Rightarrow$ single-source.
- **Single-pair shortest path**
  - Find a shortest path from u to v for given vertices u and v.
- **All-pairs shortest-paths**
  - Find a shortest path from u to v for every pair of vertices u and v.

Negative Weight Edges & Cycles

- Negative-weight edges may form negative-weight cycles
- If such cycles are reachable from the source, then $\delta(s, v)$ is not properly defined!
- Keep going around the cycle, and get $w(s, v) = -\infty$ for all v on the cycle.

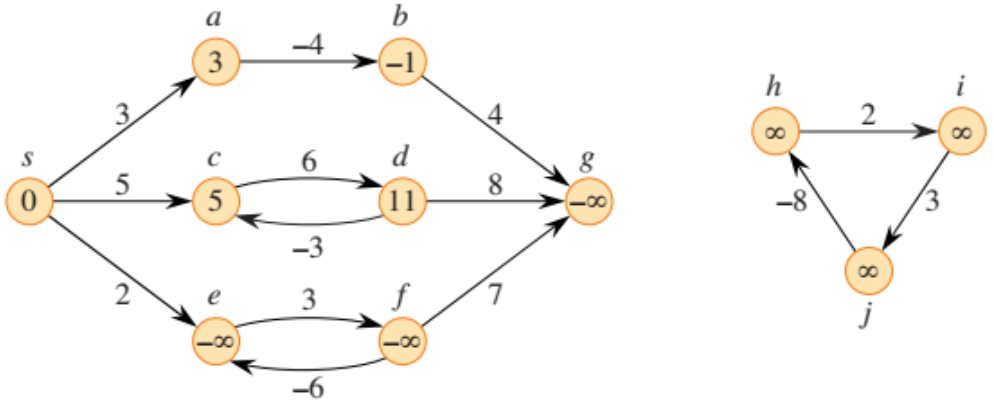


Figure 22.1 Negative edge weights in a directed graph. The shortest-path weight from source s appears within each vertex. Because vertices e and f form a negative-weight cycle reachable from s, they have shortest-path weights of $-\infty$. Because vertex g is reachable from a vertex whose shortest-path weight is $-\infty$, it, too, has a shortest-path weight of $-\infty$. Vertices such as h, i, and j are not reachable from s, and so their shortest-path weights are ∞, even though they lie on a negative-weight cycle.

Shortest Paths & Cycles

- Shortest path cannot contain *negative-weight cycle*
- It cannot contain *a positive-weight cycle* as the removal of the cycle from the path produces a path with the *same source and destination vertices* and a *lower path weight*
- Can contain *0-weight cycles*: *removal* from the path produces *another cycle-free path* with the *same weight*.
- As long as a shortest path has *0-weight cycles*, they can be *repeatedly removed from the path*
- **WLOG we assume that *shortest paths have no cycles (simple paths).***
- Since any acyclic path in a graph $G = (V, E)$ contains at most $|V|$ distinct vertices, it also contains at most $|V| - 1$ edges.
- Any shortest path contains at most $|V| - 1$ edges.

Representing Shortest Paths

- Given a graph $G=(V, E)$, maintain for each vertex $v \in V$ a predecessor $v.\pi$ that is either *another vertex* or *NIL*
- If G contains *no negative-weight cycles* reachable from the source vertex $s \in V$, so that shortest paths are *well defined*
- A *shortest-paths tree induced by* π, rooted at s is a directed subgraph $G'=(V',E')$, where $V' \subseteq V$ and $E' \subseteq E$, such that:
  - V' is the set of vertices reachable from s in G
  - G' forms a *rooted tree* with root s
  - for all $v \in V'$, the unique simple path from s to v in G' is a shortest path from s to v in G

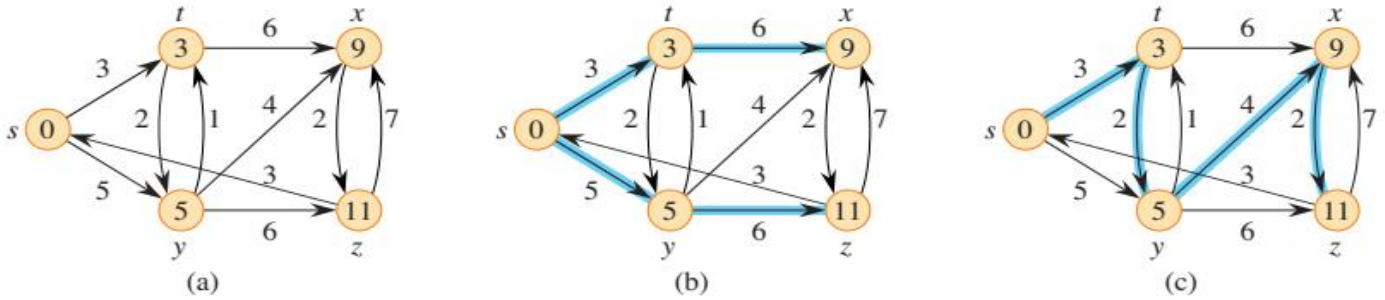


Figure 22.2 **(a)** A weighted, directed graph with shortest-path weights from source s. **(b)** The blue edges form a shortest-paths tree rooted at the source s. **(c)** Another shortest-paths tree with the same root.

- *Shortest paths are not necessarily unique, and neither are the shortest path trees.*

Shortest Path Algorithms

- Dijkstra's algorithm
  - Negative weights are not allowed.
- Bellman-Ford algorithm
  - Negative weights are allowed
  - Negative cycles reachable from the source are not allowed.
- Operations common in both algorithms:
  - Initialization
  - Relaxation

Initialization & Relaxation

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0
```

- Initializes the shortest-path estimates and predecessors for all $v \in V - \{s\}$
- *v.d: shortest-path estimate* (an upper bound on the weight of a shortest path from source *s* to *v*)
- Running time complexity $\Theta(V)$

```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2      v.d = u.d + w(u, v)
3      v.π = u
```

- The process of *relaxing* an edge *(u, v)* consists of testing whether going through vertex *u* improves the shortest path to vertex *v* found so far
- If found, update *v.d* and *v.*Π
- Running time complexity *O (1)*

Initialization & Relaxation

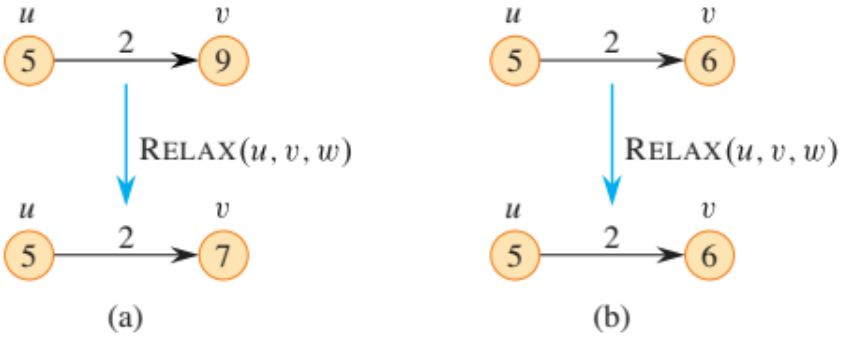


Figure 22.3 Relaxing an edge (u, v) with weight $w(u, v) = 2$. The shortest-path estimate of each vertex appears within the vertex. **(a)** Because $v.d > u.d + w(u, v)$ prior to relaxation, the value of $v.d$ decreases. **(b)** Since we have $v.d \leq u.d + w(u, v)$ before relaxing the edge, the relaxation step leaves $v.d$ unchanged.

- Each single-source shortest-paths algorithm calls *INITIALIZE-SINGLE-SOURCE* and then repeatedly relaxes edges.
- *Relaxation* is the only means by which *shortest path estimates* and *predecessors change*.
- The algorithms differ in how many times they relax each edge and the order in which they relax edges.
- *Dijkstra's algorithm* and the shortest-paths algorithm for directed acyclic graphs relax each edge *exactly once*.
- The *Bellman-Ford algorithm* relaxes each edge $|V| - 1$ times.

The Bellman-Ford Algorithm

- It solves the single-source shortest-paths problem in the general case in which edge weights may be negative.
- The algorithm returns a Boolean value indicating whether there is a *negative-weight cycle* that is reachable from the source.
- If there is such a cycle, the algorithm indicates that no solution exists (checks in lines 5-7).
- If there is no such cycle, the algorithm produces the shortest paths and their weights.

```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i = 1 to |G.V| − 1
3       for each edge (u, v) ∈ G.E
4            RELAX(u, v, w)
5  for each edge (u, v) ∈ G.E
6       if v.d > u.d + w(u, v)
7            return FALSE
8  return TRUE
```

Dijkstra's Algorithm

- It solves the single-source shortest-paths problem on weighted directed graph $G = (V, E)$, but requires edge weights to be nonnegative: $w(u, v) >= 0$, *for all* $(u, v) \in G$
- Generalizes breadth-first search to weighted graphs.
- Dijkstra's algorithm maintains a set S of vertices whose final shortest path weights from the source s have already been determined.
- The algorithm repeatedly selects the vertex $u \in V\text{-}S$ with the minimum shortest-path estimate, adds u into S, and relaxes all edges leaving u.
- The procedure *DIJKSTRA* replaces the *first-in, first-out queue* of breadth-first search by a *min-priority queue* Q of vertices, keyed by their d values.

```
DIJKSTRA(G, w, s)
 1  INITIALIZE-SINGLE-SOURCE(G, s)
 2  S = ∅
 3  Q = ∅
 4  for each vertex u ∈ G.V
 5      INSERT(Q, u)
 6  while Q ≠ ∅
 7      u = EXTRACT-MIN(Q)
 8      S = S ∪ {u}
 9      for each vertex v in G.Adj[u]
10          RELAX(u, v, w)
11          if the call of RELAX decreased v.d
12              DECREASE-KEY(Q, v, v.d)
```

Properties of Shortest Paths and Relaxation

Triangle inequality (Lemma 22.10)
For any edge $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

Upper-bound property (Lemma 22.11)
We always have $v.d \geq \delta(s, v)$ for all vertices $v \in V$, and once $v.d$ achieves the value $\delta(s, v)$, it never changes.

No-path property (Corollary 22.12)
If there is no path from s to v, then we always have $v.d = \delta(s, v) = \infty$.

Convergence property (Lemma 22.14)
If $s \rightsquigarrow u \rightarrow v$ is a shortest path in G for some $u, v \in V$, and if $u.d = \delta(s, u)$ at any time prior to relaxing edge (u, v), then $v.d = \delta(s, v)$ at all times afterward.

Path-relaxation property (Lemma 22.15)
If $p = \langle v_0, v_1, \ldots, v_k \rangle$ is a shortest path from $s = v_0$ to v_k, and the edges of p are relaxed in the order $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$, then $v_k.d = \delta(s, v_k)$. This property holds regardless of any other relaxation steps that occur, even if they are intermixed with relaxations of the edges of p.

Predecessor-subgraph property (Lemma 22.17)
Once $v.d = \delta(s, v)$ for all $v \in V$, the predecessor subgraph is a shortest-paths tree rooted at s.

All Pair Shortest Path Problem

- **The problem:** find the shortest path between every pair of vertices of a graph.
- **The graph:** may contain negative edges but no negative cycles
- **A representation:** a weight matrix where
  - $W(i,j) = 0$ if $i = j$
  - $W(i,j) = \infty$ if there is no edge between i and j.
  - $W(i,j) =$ "weight of edge".
- The all-pairs shortest-paths problem may be solved by running a single-source shortest-paths algorithm $|V|$ *times*, once with each vertex as the source.
- If the weights are nonnegative, *Dijkstra's Algorithm* can be used.
  - Implementation:
    - min-priority queue with a linear array: $O(V^3 + V^2E)$.
    - binary min-heap implementation of the min-priority queue: $O(V(V+E) \lg V)$
      - For sparse graphs $E = \Omega(V)$, running time $\approx O(V^2 \lg V)$
      - For dense graphs $E = O(V^2)$, running time $\approx O(V^3 \lg V)$
- If the graph contains *negative-weight edges*, we can run the slower *Bellman-Ford algorithm* once from each vertex.
  - Running time: $O(V^3 + V^2E)$, which on a dense graph $\sim O(V^4)$.

Recursive Solution to All Pairs Shortest Paths Problem

- Let $l^{(r)}_{ij}$ be the minimum weight of any path from vertex i to vertex j that contains at most r edges. When $r = 0$, there is a shortest path from i to j with no edges if and only if $i = j$, yielding

$$l_{ij}^{(0)} = \begin{cases} 0 & \text{if } i = j \ , \\ \infty & \text{if } i \neq j \ . \end{cases}$$

- For $r \geq 1$, one way to achieve *a minimum-weight path* from i to j with at most r edges is by taking a path of at most $r - 1$ edges, *so that* $l^{(r)}_{ij} = l^{(r-1)}_{ij}$.
- Another way is by taking a path of at most $r-1$ edges from i to some vertex k and then taking the edge (k, j), so that

$$l_{ij}^{(r)} = l_{ik}^{(r-1)} + w(k, j)$$

- To examine paths from i to j consisting of at most r edges, all possible predecessors k of j can be checked, giving the recursive definition

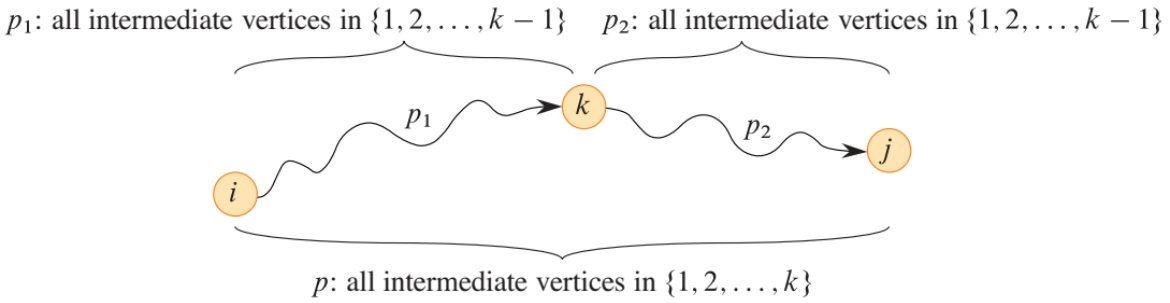
$$\begin{aligned} l_{ij}^{(r)} &= \min \left\{ l_{ij}^{(r-1)}, \ \min \left\{ l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n \right\} \right\} \\ &= \min \left\{ l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n \right\} \ . \end{aligned}$$

  - Both negative-weight and positive-weight cycles have no effect on the shortest path from vertex i to vertex j
  - Any simple path contains at most $n - 1$ edges, a path from vertex i to vertex j with more than $n-1$ edges (when the path has cycles) cannot have lower weight than a shortest path from i to j
  - The actual shortest-path weights are therefore given by

$$\delta(i, j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} = \cdots \ .$$

The Floyd-Warshall Algorithm

- Structure of the Shortest Path
  - The algorithm considers the *intermediate vertices of a shortest path*, where an *intermediate vertex* of a simple path $p = \{v_1, v_2, \ldots v_l\}$ is any vertex of p other than v_1 or v_l in the set $\{v_2, v_3, \ldots, v_{l-1}\}$
  - Ordering of the vertices of G by $V = \{1, 2, \ldots, n\}$, take a subset $\{1, 2, \ldots, k\}$ of vertices for some $1 \le k \le n$.
  - For any pair of vertices $i, j \in V$, consider all paths from i to j whose intermediate vertices are all drawn from $\{1, 2, \ldots, k\}$, and let p be a *minimum-weight path* from among them.
  - The Floyd-Warshall algorithm exploits a relationship between path p and shortest paths from i to j with all intermediate vertices in the set $\{1, 2, \ldots, k\text{-}1\}$.
  - The details of the relation depends on if k is an intermediate vertex of path p or not.



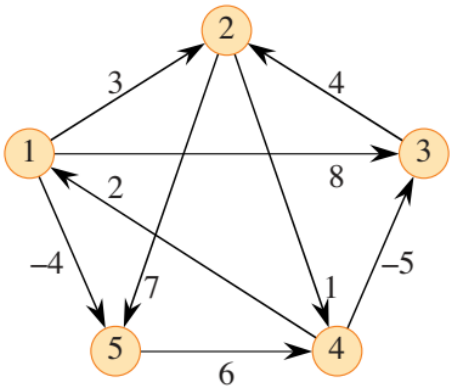
- A recursive solution to the all-pairs shortest-paths problem:

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0\,, \\ \min\left\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right\} & \text{if } k \ge 1\,. \end{cases}$$

Because for any path, all intermediate vertices belong to the set $\{1, 2, \ldots, n\}$, the matrix $D^{(n)} = \left(d_{ij}^{(n)}\right)$ gives the final answer: $d_{ij}^{(n)} = \delta(i, j)$ for all $i, j \in V$.

The Floyd-Warshall Algorithm

```
FLOYD-WARSHALL(W, n)
1  D^(0) = W
2  for k = 1 to n
3      let D^(k) = (d_ij^(k)) be a new n × n matrix
4      for i = 1 to n
5          for j = 1 to n
6              d_ij^(k) = min {d_ij^(k-1), d_ik^(k-1) + d_kj^(k-1)}
7  return D^(n)
```



- The running time of the algorithm is determined by the *triply nested for loops* of lines 2-6.
- Each execution of line 6 takes $O(1)$ time, the algorithm runs in $\Theta(n^3)$ *time*.
- The code is tight, with *no elaborate data structures*, and so the constant hidden in the Θ*-notation* is *small*.
- The algorithm is *quite practical for even moderate-sized input graphs*.